

LAPORAN PRAKTIKUM PEKAN 4

STRUKTUR DATA

Disusun Oleh:

Muhammad Fharel

2511531010

Dosen Pengampu:

Dr. Wahyudi S.T.M.T

Asisten Pratikum:

Muhammad Galid Avero



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Segala puji penulis panjatkan ke hadirat Allah SWT atas rahmat dan karunia-Nya sehingga laporan praktikum Struktur Data pada tanggal 28 April 2026 yang membahas tentang konsep dasar struktur data *Queue* serta implementasinya menggunakan *array* dan *class* bawaan Java seperti *LinkedList*. Selain itu, memahami berbagai operasi dasar *Queue* seperti *enqueue*, *dequeue*, dan *peek*. Materi ini penting karena menjadi fondasi dalam memahami struktur data.

Ucapan terima kasih ditujukan kepada dosen pengampu, asisten praktikum, serta rekan-rekan yang telah membantu dalam proses pelaksanaan praktikum. Penulis menyadari bahwa penulisan laporan masih jauh dari kata sempurna, oleh karena itu kritik dan saran sangat diharapkan untuk penyempurnaan di kemudian hari. Semoga laporan ini memberikan manfaat dan menambah wawasan pembaca.

Padang, 29 April 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan Praktikum	1
1.3 Manfaat Praktikum	1
BAB II PEMBAHASAN.....	2
2.1 Queue.....	2
2.2 Kode Pemograman	2
2.2.1 Class QueueArray	2
2.2.2 Class QueueArray Driver	4
2.2.3 Output QueueArray Driver	5
2.2.4 Class ReverseData.....	5
2.2.5 Output ReverseData	6
2.2.6 Class QueueLinkedList	6
2.2.7 Output QueueLinkedList.....	7
2.2.8 Class IterasiQueue.....	7
2.2.9 Output IterasiQueue	8
BAB III PENUTUP	9
3.1 Kesimpulan.....	9
DAFTAR PUSTAKA.....	10
TUGAS PRAKTIKUM.....	11

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pemrograman, pengolahan data yang terstruktur sangat penting untuk mempermudah proses penyimpanan dan pengambilan data. Salah satu struktur data yang digunakan untuk mengatur urutan data adalah Queue. Queue merupakan struktur data yang bekerja dengan prinsip FIFO (First In First Out), yaitu data yang pertama masuk akan menjadi data pertama yang keluar.

Queue banyak digunakan dalam kehidupan sehari-hari, seperti antrian layanan, sistem pemrosesan data, dan manajemen tugas dalam sistem komputer. Dalam Java, Queue dapat diimplementasikan menggunakan array maupun menggunakan class bawaan seperti LinkedList. Pada praktikum ini, dilakukan beberapa implementasi Queue untuk memahami cara kerja serta penerapannya dalam program Java.

1.2 Tujuan Praktikum

1. Memahami konsep dasar struktur data Queue.
2. Mengetahui prinsip kerja FIFO.
3. Mampu mengimplementasikan Queue menggunakan array dan LinkedList.
4. Memahami operasi dasar Queue (enqueue, dequeue, peek).
5. Menerapkan Queue dalam program sederhana.

1.3 Manfaat Praktikum

1. Memahami cara kerja Queue dengan prinsip FIFO.
2. Melatih penggunaan operasi enqueue dan dequeue dalam program.
3. Mengetahui perbedaan implementasi Queue menggunakan array dan LinkedList.
4. Menjadi dasar untuk memahami sistem antrian dalam aplikasi nyata.

BAB II

PEMBAHASAN

2.1 Queue

Queue adalah struktur data yang bekerja dengan prinsip First In First Out (FIFO), yaitu data yang pertama masuk akan menjadi data pertama yang keluar. Konsep ini mirip dengan antrian di kehidupan sehari-hari, di mana orang yang datang lebih dulu akan dilayani terlebih dahulu.

Dalam Java, Queue dapat diimplementasikan menggunakan array maupun menggunakan class bawaan seperti LinkedList. Operasi utama dalam Queue adalah enqueue untuk menambahkan data ke antrian, dequeue untuk menghapus data dari depan, dan peek untuk melihat elemen terdepan tanpa menghapusnya.

Melalui praktikum ini, dapat dipahami bahwa Queue sangat berguna untuk mengelola data yang diproses secara berurutan, serta dapat digunakan dalam berbagai kasus seperti sistem antrian dan pengolahan data.

2.2 Kode Pemograman

2.2.1 Class QueueArray

```
2 package pekan4_2511531010;
3
4 public class QueueArray_2511531010 {
5     int front_1010, rear_1010, size_1010;
6     int capacity_1010;
7     int array_1010[];
8
9     public QueueArray_2511531010(int capacity_1010) {
10         this.capacity_1010 = capacity_1010;
11         front_1010 = this.size_1010= 0;
12         rear_1010 = capacity_1010 - 1;
13         array_1010 = new int[this.capacity_1010];
14     }
15
16     boolean isFull_1010(QueueArray_2511531010 queue) {
17         return (queue.size_1010 == queue.capacity_1010);
18     }
19 }
```

```

20 boolean isEmpty_1010(QueueArray_2511531010 queue) {
21     return (queue.size_1010 == 0);
22 }
23
24 void enqueue_1010(int item_1010) {
25     if (isFull_1010(this))
26         return;
27     this.rear_1010 =(this.rear_1010 + 1) % this.capacity_1010;
28     this.array_1010[this.rear_1010] = item_1010;
29     this.size_1010=this.size_1010 + 1;
30     System.out.println(item_1010 + " enqueue to queue");
31 }
32
33 int dequeue_1010() {
34     if (isEmpty_1010(this))
35         return Integer.MIN_VALUE;
36     int item_1010 = this.array_1010[this.front_1010];
37     this.front_1010 = (this.front_1010 + 1) %
this.capacity_1010;
38     this.size_1010 = this.size_1010 - 1;
39     return item_1010;
40 }
41 int front_1010() {
42     if (isEmpty_1010(this))
43         return Integer.MIN_VALUE;
44
45     return this.array_1010 [this.front_1010];
46 }
47 int rear_1010() {
48     if (isEmpty_1010(this))
49         return Integer.MIN_VALUE;
50
51     return this.array_1010 [this.rear_1010];
52 }
53 // mencetak elemen antrian
54 void display_1010() {
55     int i;
56     if (front_1010 == rear_1010) {
57         System.out.printf("\nAntrian Kosong\n");
58         return;
59     }
60     //kunjungi dari belakang dan cetak
61     for (i = front_1010; i < rear_1010; i++) {
62         System.out.printf(" %d <-- ", array_1010[i]);
63     }
64     return;
65 }
66 }

```

Gambar 2.1

Variabel front, rear, dan size digunakan untuk mengatur posisi dan jumlah data dalam antrian. Method enqueue digunakan untuk menambah data ke belakang antrian, sedangkan dequeue untuk

menghapus data dari depan. Selain itu, terdapat method untuk melihat elemen depan dan belakang, serta menampilkan seluruh isi antrian.

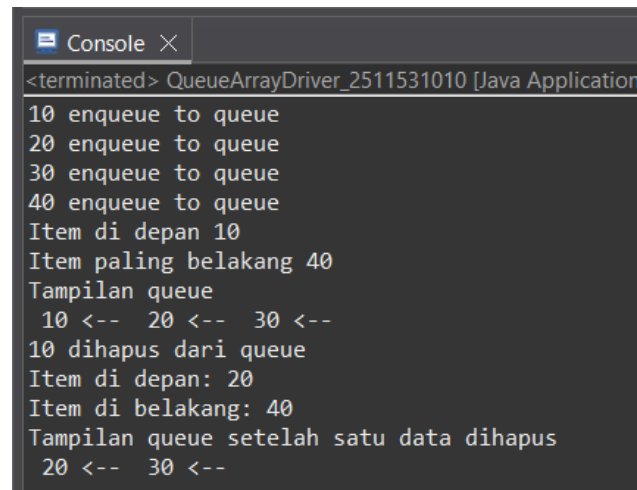
2.2.2 Class QueueArray Driver

```
2 package pekan4_2511531010;
3
4 public class QueueArrayDriver_2511531010 {
5
6     public static void main(String[] args) {
7         QueueArray_2511531010 queue_1010 = new
8         QueueArray_2511531010(1000);
9         queue_1010.enqueue_1010(10);
10        queue_1010.enqueue_1010(20);
11        queue_1010.enqueue_1010(30);
12        queue_1010.enqueue_1010(40);
13        System.out.println("Item di depan " +
14        queue_1010.front_1010());
15        System.out.println("Item paling belakang " +
16        queue_1010.rear_1010());
17        System.out.println("Tampilan queue");
18        queue_1010.display_1010();
19        System.out.println();
20        System.out.println(queue_1010.dequeue_1010() + " dihapus
21        dari queue");
22        System.out.println("Item di depan: " +
23        queue_1010.front_1010());
24        System.out.println("Item di belakang: " +
25        queue_1010.rear_1010());
26        System.out.println("Tampilan queue setelah satu data
27        dihapus");
28        queue_1010.display_1010();
29    }
30 }
```

Gambar 2.2

Program melakukan beberapa operasi seperti menambahkan data, melihat elemen depan dan belakang, serta menghapus satu data dari antrian, lalu menampilkan kembali kondisi antrian.

2.2.3 Output QueueArray Driver

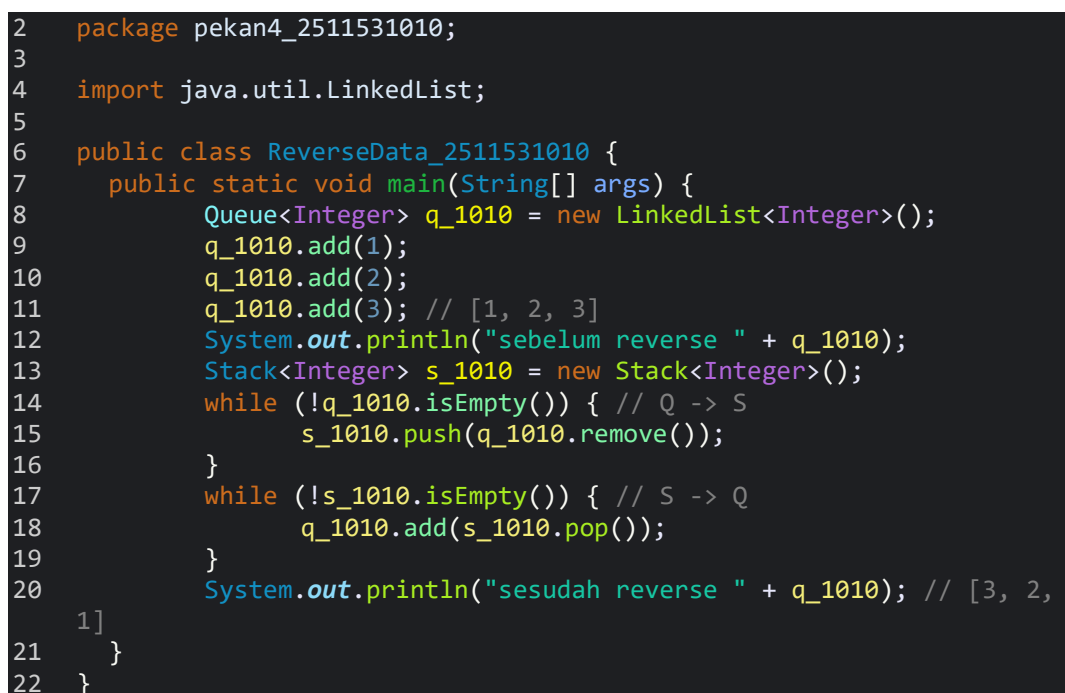


```
<terminated> QueueArrayDriver_2511531010 [Java Application]
10 enqueue to queue
20 enqueue to queue
30 enqueue to queue
40 enqueue to queue
Item di depan 10
Item paling belakang 40
Tampilan queue
10 <-- 20 <-- 30 <--
10 dihapus dari queue
Item di depan: 20
Item di belakang: 40
Tampilan queue setelah satu data dihapus
20 <-- 30 <--
```

Gambar 2.3

Output menampilkan data yang dimasukkan ke dalam antrian, kemudian menunjukkan elemen paling depan dan belakang. Setelah dilakukan penghapusan, output akan memperlihatkan data yang dihapus serta perubahan isi antrian, sehingga terlihat bagaimana urutan data bergeser setelah dequeue dilakukan.

2.2.4 Class ReverseData

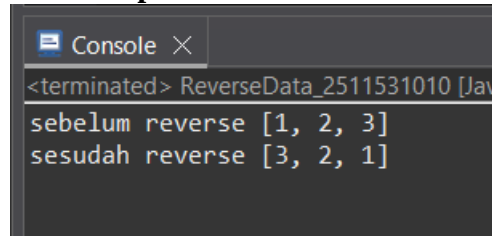


```
2 package pekan4_2511531010;
3
4 import java.util.LinkedList;
5
6 public class ReverseData_2511531010 {
7     public static void main(String[] args) {
8         Queue<Integer> q_1010 = new LinkedList<Integer>();
9         q_1010.add(1);
10        q_1010.add(2);
11        q_1010.add(3); // [1, 2, 3]
12        System.out.println("sebelum reverse " + q_1010);
13        Stack<Integer> s_1010 = new Stack<Integer>();
14        while (!q_1010.isEmpty()) { // Q -> S
15            s_1010.push(q_1010.remove());
16        }
17        while (!s_1010.isEmpty()) { // S -> Q
18            q_1010.add(s_1010.pop());
19        }
20        System.out.println("sesudah reverse " + q_1010); // [3, 2,
21        1]
22    }
23 }
```

Gambar 2.4

Class ini digunakan untuk membalik urutan data dalam Queue dengan bantuan Stack. Data dipindahkan dari Queue ke Stack, lalu dikembalikan lagi ke Queue sehingga urutannya menjadi terbalik.

2.2.5 Output ReverseData



Gambar 2.5

Awalnya data ditampilkan dalam urutan normal, kemudian setelah proses reverse, urutannya menjadi terbalik. Hal ini menunjukkan bahwa kombinasi Queue dan Stack dapat digunakan untuk mengubah urutan data.

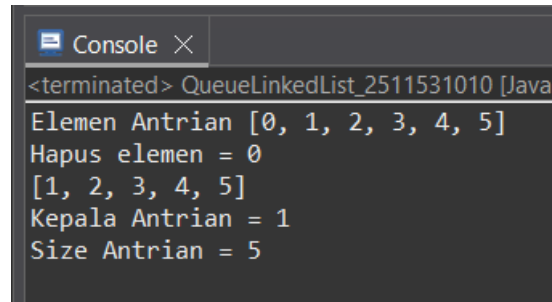
2.2.6 Class QueueLinkedList

```
2 package pekan4_2511531010;
3
4 import java.util.LinkedList;
5
6 public class QueueLinkedList_2511531010 {
7     public static void main(String[] args) {
8         Queue<Integer> q_1010 = new LinkedList<>();
9         // tambah elemen 0,1,2,3,4,5 ke antrian
10        for (int i_1010 = 0; i_1010 < 6; i_1010++) {
11            q_1010.add(i_1010);
12        }
13        // menampilkan isi antrian
14        System.out.println("Elemen Antrian " + q_1010);
15        // untuk menghapus kepala antrian
16        int hapus_1010 = q_1010.remove();
17        System.out.println("Hapus elemen = " + hapus_1010);
18        System.out.println(q_1010);
19        // untuk melihat antrian terdepan
20        int depan_1010 = q_1010.peek();
21        System.out.println("Kepala Antrian = " + depan_1010);
22
23        int banyak_1010 = q_1010.size();
24        System.out.println("Size Antrian = " + banyak_1010);
25    }
26 }
```

Gambar 2.6

Program menambahkan beberapa elemen, menghapus elemen pertama, melihat elemen terdepan, dan menampilkan jumlah elemen dalam antrian.

2.2.7 Output QueueLinkedList



```
<terminated> QueueLinkedList_2511531010 [Java]
Elemen Antrian [0, 1, 2, 3, 4, 5]
Hapus elemen = 0
[1, 2, 3, 4, 5]
Kepala Antrian = 1
Size Antrian = 5
```

Gambar 2.7

Output menampilkan isi awal antrian, kemudian menunjukkan elemen yang dihapus dari depan. Setelah itu, ditampilkan kondisi antrian terbaru, elemen terdepan, serta jumlah elemen yang tersisa, sehingga terlihat perubahan data setelah dilakukan operasi.

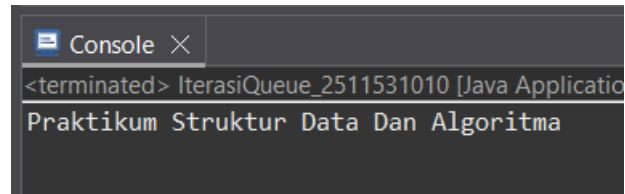
2.2.8 Class IterasiQueue

```
2 package pekan4_2511531010;
3
4 import java.util.Iterator;
5
6 public class IterasiQueue_2511531010 {
7     public static void main(String args[]) {
8
9         Queue<String> q_1010 = new LinkedList<>();
10
11         q_1010.add("Praktikum");
12         q_1010.add("Struktur");
13         q_1010.add("Data");
14         q_1010.add("Dan");
15         q_1010.add("Algoritma");
16         Iterator<String> iterator_1010 = q_1010.iterator();
17         while (iterator_1010.hasNext()) {
18             System.out.print(iterator_1010.next() + " ");
19         }
20     }
21 }
22
```

Gambar 2.8

Menampilkan isi Queue menggunakan Iterator. Data berupa kata dimasukkan ke dalam antrian, lalu ditampilkan satu per satu menggunakan perulangan.

2.2.9 Output IterasiQueue



Gambar 2.9

Output menampilkan seluruh elemen dalam queue secara berurutan dalam satu baris. Hal ini menunjukkan bahwa data dalam queue dapat diakses dan ditampilkan satu per satu sesuai urutan penyimpanannya.

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data Queue merupakan metode pengelolaan data yang bekerja dengan prinsip First In First Out (FIFO), di mana data yang pertama masuk akan menjadi data pertama yang keluar. Konsep ini sangat cocok digunakan dalam berbagai kasus yang membutuhkan sistem antrian, seperti pemrosesan data dan layanan berurutan.

Melalui beberapa implementasi yang telah dibuat, baik menggunakan array maupun class bawaan Java seperti LinkedList, dapat dipahami bahwa Queue memiliki fleksibilitas dalam penerapannya. Operasi dasar seperti enqueue, dequeue, dan peek menjadi inti dalam pengelolaan data pada Queue dan sangat penting untuk dipahami.

Selain itu, queue dapat dikombinasikan dengan struktur data lain, seperti Stack, untuk menyelesaikan masalah tertentu, misalnya dalam proses membalik urutan data. Hal ini membuktikan bahwa pemahaman terhadap struktur data tidak hanya penting secara teori, tetapi juga sangat berguna dalam praktik pemrograman.

DAFTAR PUSTAKA

- [1] Oracle Corporation. 2023. *Queue Interface (Java Platform SE)*. Diakses dari <https://docs.oracle.com/javase/8/docs/api/java/util/Queue.html>

TUGAS PRAKTIKUM

Soal:

1. Deskripsi Tugas

Mahasiswa diminta untuk mengimplementasikan **struktur data Queue (Antrian)** menggunakan **array dalam bahasa Java**.

Studi kasus yang digunakan adalah **antrian pelanggan di loket pelayanan**. Setiap pelanggan yang datang akan masuk ke dalam antrian dan akan dilayani berdasarkan prinsip **FIFO (First In First Out)**.

Program harus mampu:

- Menambahkan pelanggan ke antrian (**enqueue**)
- Menghapus pelanggan dari antrian (**dequeue**)
- Menampilkan isi antrian
- Mengecek kondisi antrian (kosong atau penuh)

2. Aturan Penamaan (Wajib Dipatuhi)

1. Nama Kelas

Gunakan akhiran **NIM lengkap**

Contoh:

- `AntrianLoket_2311532008`
- `Queue_2311532008`

1. Nama File Java

Harus sama dengan nama class

Contoh:

`AntrianLoket_2311532008.java`

2. Penamaan Variabel (Gunakan standar berikut):

- `front` → indeks depan

- `rear` → indeks belakang
 - `max` → kapasitas array
 - `queue[]` → array penyimpan data
-

3. Struktur Tugas

Program minimal memiliki:

a. Deklarasi dan Inisialisasi

- Array untuk menyimpan data antrian
 - Variabel `front` dan `rear`
 - Kapasitas maksimal queue
-

b. Method yang Wajib Ada

1. **enqueue()**
 - Menambahkan data ke belakang antrian
 - Jika penuh → tampilkan pesan
2. **dequeue()**
 - Menghapus data dari depan antrian
 - Jika kosong → tampilkan pesan
3. **display()**
 - Menampilkan seluruh isi antrian
4. **isEmpty()**
 - Mengembalikan nilai true jika kosong
5. **isFull()**
 - Mengembalikan nilai true jika penuh
6. **Reverse()**
 - Membalik seluruh isi antrian

Kode Pemograman:

1. Class AntrianLoket_2511531010

```
2. package pekan4_2511531010;
3.
4. public class AntrianLoket_2511531010 {
5.
6.     String queue_1010[];
7.     int front_1010, rear_1010, max_1010;
8.
9.     // constructor
10.    public AntrianLoket_2511531010(int max_1010) {
11.        this.max_1010 = max_1010;
12.        queue_1010 = new String[max_1010];
13.        front_1010 = 0;
14.        rear_1010 = -1;
15.    }
16.
17.    boolean isEmpty_1010() {
18.        return (rear_1010 < front_1010);
19.    }
20.
21.    boolean isFull_1010() {
22.        return (rear_1010 == max_1010 - 1);
23.    }
24.
25.    void enqueue_1010(String data_1010) {
26.        if (isFull_1010()) {
27.            System.out.println("Antrian penuh!");
28.            return;
29.        }
30.        queue_1010[++rear_1010] = data_1010;
31.        System.out.println("Data berhasil ditambahkan ke
antrian");
32.    }
33.
34.    void dequeue_1010() {
35.        if (isEmpty_1010()) {
36.            System.out.println("Antrian kosong!");
37.            return;
38.        }
39.        System.out.println(queue_1010[front_1010] + " telah
dilayani");
40.        front_1010++;
41.    }
42.
43.    void display_1010() {
44.        if (isEmpty_1010()) {
45.            System.out.println("Antrian kosong!");
46.            return;
47.        }
48.
49.        System.out.println("Isi antrian:");
50.        int no_1010 = 1;
```

```

51.         for (int i_1010 = front_1010; i_1010 <= rear_1010;
52.             i_1010++) {
53.             System.out.println(no_1010 + ". " +
54.                 queue_1010[i_1010]);
55.             no_1010++;
56.         }
57.     void reverse_1010() {
58.         if (isEmpty_1010()) {
59.             System.out.println("Antrian kosong!");
60.             return;
61.         }
62.
63.         int start_1010 = front_1010;
64.         int end_1010 = rear_1010;
65.
66.         while (start_1010 < end_1010) {
67.             String temp_1010 = queue_1010[start_1010];
68.             queue_1010[start_1010] =
69.                 queue_1010[end_1010];
70.             queue_1010[end_1010] = temp_1010;
71.             start_1010++;
72.             end_1010--;
73.         }
74.
75.         System.out.println("Antrian berhasil dibalik");
76.     }
77. }

```

1. Class AntrianLoketDriver_2511531010

```

2. package pekan4_2511531010;
3.
4. import java.util.Scanner;
5.
6. public class AntrianLoketDriver_2511531010 {
7.
8.     public static void main(String[] args) {
9.         Scanner input_1010 = new Scanner(System.in);
10.        AntrianLoket_2511531010 antrian_1010 = new
11.            AntrianLoket_2511531010(100);
12.
13.        int pilihan_1010;
14.
15.        do {
16.            System.out.println("=== PROGRAM ANTRIAN LOKET
17.            ===");
18.            System.out.println("1. Tambah Antrian");
19.            System.out.println("2. Hapus Antrian");
20.            System.out.println("3. Tampilkan Antrian");
21.            System.out.println("4. Reverse");
22.            System.out.println("5. Keluar");

```

```

21.         System.out.print("Pilih menu: ");
22.
23.         pilihan_1010 = input_1010.nextInt();
24.         input_1010.nextLine();
25.
26.         switch (pilihan_1010) {
27.
28.             case 1:
29.                 System.out.print("Masukkan nama
pelanggan: ");
30.                 String nama_1010 =
input_1010.nextLine();
31.                 antrian_1010.enqueue_1010(nama_1010);
32.                 break;
33.
34.             case 2:
35.                 antrian_1010.dequeue_1010();
36.                 break;
37.
38.             case 3:
39.                 antrian_1010.display_1010();
40.                 break;
41.
42.             case 4:
43.                 antrian_1010.reverse_1010();
44.                 antrian_1010.display_1010();
45.                 break;
46.
47.             case 5:
48.                 System.out.println("Program
selesai");
49.                 break;
50.
51.             default:
52.                 System.out.println("Pilihan
tidak valid!");
53.         }
54.
55.     } while (pilihan_1010 != 5);
56.
57.     input_1010.close();
58. }
59. }

```

3. Output

```
Console X
<terminated> AntrianLoketDriver_2511531010 [Java]
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: FHAREL
Data berhasil ditambahkan ke antrian
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: JENNIE
Data berhasil ditambahkan ke antrian
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. FHAREL
2. JENNIE
```

```

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 4
Antrian berhasil dibalik
Isi antrian:
1. JENNIE
2. FHAREL
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 2
JENNIE telah dilayani
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. FHAREL
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 5
Program selesai

```

Program ini dibuat untuk menerapkan struktur data Queue (antrian) menggunakan array dengan konsep FIFO (First In First Out), yaitu data yang masuk lebih dulu akan keluar lebih dulu. Contohnya seperti antrian di loket, di mana pelanggan dilayani sesuai urutan kedatangan.

Program memiliki class `AntrianLoket_2511531010` yang mengatur data menggunakan variabel `front`, `rear`, `max`, dan array sebagai penyimpanan. Terdapat beberapa method utama seperti `enqueue` untuk menambah data, `dequeue` untuk menghapus data depan, `display` untuk menampilkan antrian, `isEmpty` dan `isFull` untuk mengecek kondisi, serta `reverse` untuk membalik

urutan data. Program juga dilengkapi menu agar pengguna bisa memilih operasi dengan mudah. Secara keseluruhan, program ini menunjukkan cara kerja antrian secara sederhana dan jelas menggunakan Java.